Integration Manual

for S32K3 CAN Driver

Document Number: IM34CANASR4.4 Rev0000R2.0.0 Rev. 1.0

1 Revision History 2

2 Introduction 3

2.1 Supported Derivatives . . . 3
2.2 Overview . . . 4
2.3 About This Manual . . . 4
2.4 Acronyms and Definitions . . . 5
2.5 Reference List . . . 5

3 Building the driver 6

3.1 Build Options . . . 6
3.1.1 GCC Compiler/Assembler/Linker Options . . . 6
3.1.2 GHS Compiler/Assembler/Linker Options . . . 9
3.1.3 IAR Compiler/Assembler/Linker Options . . . 11
3.2 Files required for compilation . . . 13
3.3 Setting up the plugins . . . 14
3.3.1 Location of various files inside the CAN module folder . . . 14

4 Function calls to module 15

4.1 Function Calls during Start-up . . . 15
4.2 Function Calls during Shutdown . . . 15
4.3 Function Calls during Wake-up . . . 15

5 Module requirements 16

5.1 Exclusive areas to be defined in BSW scheduler . . . 16
5.2 Exclusive areas not available on this platform . . . 24
5.3 Peripheral Hardware Requirements . . . 24
5.4 ISR to configure within AutosarOS - dependencies . . . 25
5.5 ISR Macro . . . 26
5.5.1 Without an Operating System . . . 26
5.5.2 With an Operating System . . . 26
5.6 Other AUTOSAR modules - dependencies . . . 26
5.7 Data Cache Restrictions . . . 27
5.8 User Mode support . . . 27
5.8.1 User Mode configuration in the module . . . 27
5.8.2 User Mode configuration in AutosarOS . . . 29
5.9 Multicore support . . . 30

6 Main API Requirements 32

6.1 Main function calls within BSW scheduler . . . 32
6.2 API Requirements . . . 33
6.3 Calls to Notification Functions, Callbacks, Callouts . . . 33

7 Memory allocation **34**
7.1 Sections to be defined in Can_MemMap.h 34
7.2 Linker command file 35

8 Integration Steps **36**

9 External assumptions for driver **37**

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	31.03.2022	NXP RTD Team	Prepared for release RTD S32K3 2.0.0

Chapter 2

Introduction

- Supported Derivatives
- Overview
- About This Manual
- Acronyms and Definitions
- Reference List

This Integration Manual describes NXP Semiconductor AUTOSAR CAN for S32K3XX. AUTOSAR CAN driver configuration parameters and deviations from the specification are described in Driver chapter of this document. AUTOSAR CAN driver requirements and APIs are described in the AUTOSAR CAN driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32k344_mapbga257
- s32k344_mqfp172
- s32k324_mapbga257
- s32k324_mqfp172
- s32k314_mapbga257
- s32k314_mqfp172
- s32k312_mqfp172
- s32k312_mqfp100
- s32k322_mqfp172
- s32k322_mqfp100

- s32k342_mqfp172
- s32k342_mqfp100
- s32k341_mqfp172
- s32k341_mqfp100

All of the above microcontroller devices are collectively named as S32K3.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
C/CPP	C and C++ Source Code
CS	Chip Select
CTU	Cross Trigger Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Signifigant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	Specification of CAN Driver	AUTOSAR 4.4.0
2	S32K3xx Reference Manual	S32K3xx Reference Manual, Rev. 3, 10/2021
3	Errata	S32K342, Mask Set Errata for Mask 0P97C, Rev. 13 Dec 2021
		S32K312, Mask Set Errata for Mask 0P09C, Rev. 10/Jan/2022
		S32K344, Mask Set Errata for Mask 0P55A/1P55A, Rev. 30/Sep/2021
4	Datasheet	S32K3xx Data Sheet, Rev. 3 — 10/2021

Chapter 3

Building the driver

- Build Options
- Files required for compilation
- Setting up the plugins

This section describes the source files and various compilers, linker options used for building the driver.
It also explains the EB Tresos Studio plugin setup procedure.

3.1 Build Options

- GCC Compiler/Assembler/Linker Options
- GHS Compiler/Assembler/Linker Options
- IAR Compiler/Assembler/Linker Options

The RTD driver files are compiled using:

- NXP GCC 10.2.0 20200723 (Build 1728 Revision g5963bc8)
- IAR ANSI C/C++ Compiler V8.50.10.290/W32 for ARM Functional Safety
- Green Hills Multi 7.1.6d / Compiler 2021.1.4

The compiler, assembler, and linker flags used for building the driver are explained below.
The TS_T40D34M20I0R0 part of the plugin name is composed as follows:

- T = Target_Id (e.g. T40 identifies Cortex-M architecture)
- D = Derivative_Id (e.g. D34 identifies S32K3 platform)
- M = SW_Version_Major and SW_Version_Minor
- I = SW_Version_Patch
- R = Reserved

3.1.1 GCC Compiler/Assembler/Linker Options

3.1.1.1 GCC Compiler Options

Compiler Option	Description
-mcpu=cortex-m7	Targeted ARM processor for which GCC should tune the performance of the code
-mthumb	Generates code that executes in Thumb state
-mlittle-endian	Generate code for a processor running in little-endian mode
-mfpu=fpv5-sp-d16	Specifies the floating-point hardware available on the target
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions
-std=c99	Specifies the ISO C99 base standard
-Os	Optimize for size. Enables all -O2 optimizations except those that often increase code size
-ggdb3	Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program
-Wall	Enables all the warnings about constructions that some users consider questionable, and that are easy to avoid (or modify to prevent the warning), even in conjunction with macros
-Wextra	This enables some extra warning flags that are not enabled by -Wall
-pedantic	Issue all the warnings demanded by strict ISO C. Reject all programs that use forbidden extensions. Follows the version of the ISO C standard specified by the aforementioend -std option
-Wstrict-prototypes	Warn if a function is declared or defined without specifying the argument types
-Wundef	Warn if an undefined identifier is evaluated in an #if directive. Such identifiers are replaced with zero
-Wunused	Warn whenever a function, variable, label, value, macro is unused
-Werror=implicit-function-declaration	Make the specified warning into an error. This option throws an error when a function is used before being declared
-Wsign-compare	Warn when a comparison between signed and unsigned values could produce an incorrect result when the signed value is converted to unsigned.
-Wdouble-promotion	Give a warning when a value of type float is implicitly promoted to double
-fno-short-enums	Specifies that the size of an enumeration type is at least 32 bits regardless of the size of the enumerator values.
-funsigned-char	Let the type char be unsigned by default, when the declaration does not use either signed or unsigned
-funsigned-bitfields	Let a bit-field be unsigned by default, when the declaration does not use either signed or unsigned
-fomit-frame-pointer	Omit the frame pointer in functions that dont need one. This avoids the instructions to save, set up and restore the frame pointer; on many targets it also makes an extra register available.

Compiler Option	Description
-fno-common	Makes the compiler place uninitialized global variables in the BSS section of the object file. This inhibits the merging of tentative definitions by the linker so you get a multiple-definition error if the same variable is accidentally defined in more than one compilation unit
-fstack-usage	Makes the compiler output stack usage information for the program, on a per-function basis
-fdump-ipa-all	Enables all inter-procedural analysis dumps
-c	Stop after assembly and produce an object file for each source file
-DS32K3XX	Predefine S32K3XX as a macro, with definition 1
-DS32K344	Predefine S32K344 as a macro, with definition 1
-DGCC	Predefine GCC as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DD_CACHE_ENABLE	Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initalization in source file system.↩ c under the Platform driver
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initalization in source file system.c under the Platform driver
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initalization in source file system.c under the Platform driver
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPO↩ RT as a macro, with definition 1. Allows drivers to be configured in user mode.

3.1.1.2 GCC Assembler Options

Assembler Option	Description
-Xassembler-with-cpp	Specifies the language for the following input files (rather than letting the compiler choose a default based on the file name suffix)
-mcpu=cortexm7	Targeted ARM processor for which GCC should tune the performance of the code
-mthumb	Generates code that executes in Thumb state
-c	Stop after assembly and produce an object file for each source file

3.1.1.3 GCC Linker Options

Linker Option	Description
-Wl,-Map,filename	Produces a map file
-T linkerfile	Use linkerfile as the linker script. This script replaces the default linker script (rather than adding to it)
–entry=Reset_Handler	Specifies that the program entry point is Reset_Handler

Linker Option	Description
-nostartfiles	Do not use the standard system startup files when linking
-mcpu=cortexm7	Targeted ARM processor for which GCC should tune the performance of the code
-mthumb	Generates code that executes in Thumb state
-mfpu=fpv5-sp-d16	Specifies the floating-point hardware available on the target
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions
-mlittle-endian	Generate code for a processor running in little-endian mode
-ggdb3	Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program
-lc	Link with the C library
-lm	Link with the Math library
-lgcc	Link with the GCC library

3.1.2 GHS Compiler/Assembler/Linker Options

3.1.2.1 GHS Compiler Options

Compiler Option	Description
-cpu=cortexm7	Selects target processor: Arm Cortex M7
-thumb	Selects generating code that executes in Thumb state
-fpu=vfpv5_d16	Specifies hardware floating-point using the v5 version of the VFP instruction set, with 16 double-precision floating-point registers
-fsingle	Use hardware single-precision, software double-precision FP instructions
-C99	Use (strict ISO) C99 standard (without extensions)
–ghstd=last	Use the most recent version of Green Hills Standard mode (which enables warnings and errors that enforce a stricter coding standard than regular C and C++)
-Osize	Optimize for size
–gnu_asm	Enables GNU extended asm syntax support
-dual_debug	Generate DWARF 2.0 debug information
-G	Generate debug information
-keeptempfiles	Prevents the deletion of temporary files after they are used. If an assembly language file is created by the compiler, this option will place it in the current directory instead of the temporary directory
-Wimplicit-int	Produce warnings if functions are assumed to return int
-Wshadow	Produce warnings if variables are shadowed
-Wtrigraphs	Produce warnings if trigraphs are detected
-Wundef	Produce a warning if undefined identifiers are used in #if preprocessor statements
–unsigned_chars	Let the type char be unsigned, like unsigned char

Compiler Option	Description
–unsigned_fields	Bitfelds declared with an integer type are unsigned
–no_commons	Allocates uninitialized global variables to a section and initializes them to zero at program startup
–no_exceptions	Disables C++ support for exception handling
–no_slash_comment	C++ style // comments are not accepted andgenerate errors
–prototype_errors	Controls the treatment of functions referenced or called when no prototype has been provided
–incorrect_pragma_warnings	Controls the treatment of valid #pragma directives that use the wrong syntax
-c	Stop after assembly and produce an object file for each source file
-DS32K3XX	Predefine S32K3XX as a macro, with definition 1
-DS32K344	Predefine S32K344 as a macro, with definition 1
-DGHS	Predefine GHS as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DD_CACHE_ENABLE	Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initalization in source file system.↩ c under the Platform driver
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initalization in source file system.c under the Platform driver
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initalization in source file system.c under the Platform driver
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPO↩ RT as a macro, with definition 1. Allows drivers to be configured in user mode

3.1.2.2 GHS Assembler Options

Assembler Option	Description
-cpu=cortexm7	Selects target processor: Arm Cortex M7
-preprocess_assembly_files	Controls whether assembly files with standard extensions such as .s and .asm are preprocessed
-list	Creates a listing by using the name and directory of the object file with the .lst extension
-c	Stop after assembly and produce an object file for each source file

3.1.2.3 GHS Linker Options

Linker Option	Description
-e Reset_Handler	Make the symbol Reset_Handler be treated as a root symbol and the start label of the application

Linker Option	Description
-T linker_script_file.ld	Use linker_script_file.ld as the linker script. This script replaces the default linker script (rather than adding to it)
-map	Produce a map file
-keepmap	Controls the retention of the map file in the event of a link error
-Mn	Generates a listing of symbols sorted alphabetically/numerically by address
-delete	Instructs the linker to remove functions that are not referenced in the final executable. The linker iterates to find functions that do not have relocations pointing to them and eliminates them
-ignore_debug_references	Ignores relocations from DWARF debug sections when using -delete. DWA↩ RF debug information will contain references to deleted functions that may break some third-party debuggers
-Llibrary_path	Points to library_path (the libraries location) for thumb2 to be used for linking
-larch	Link architecture specific library
-lstartup	Link run-time environment startup routines. The source code for themodules in this library is provided in the src/libstartup directory
-lind_sd	Link language-independent library, containing support routines for features such as software floating point, run-time error checking, C99 complex numbers, and some general purpose routines of the ANSI C library
-v	Prints verbose information about the activities of the linker, including the libraries it searches to resolve undefined symbols
-keep=C40_Ip_AccessCode	Avoid linker remove function C40_Ip_AccessCode from Fls module because it is not referenced explicitly
-nostartfiles	Controls the start files to be linked into the executable

3.1.3 IAR Compiler/Assembler/Linker Options

3.1.3.1 IAR Compiler Options

Compiler Option	Description
–cpu=Cortex-M7	Targeted ARM processor for which IAR should tune the performance of the code
–cpu_mode=thumb	Generates code that executes in Thumb state
–endian=little	Generate code for a processor running in little-endian mode
–fpu=FPv5-SP	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). Single-precision variant.
-e	Enables all IAR C language extensions
-Ohz	Optimize for size. the compiler will emit AEABI attributes indicating the requested optimization goal. This information can be used by the linker to select smaller or faster variants of DLIB library functions
–debug	Makes the compiler include debugging information in the object modules. Including debug information will make the object files larger

Compiler Option	Description
–no_clustering	Disables static clustering optimizations. Static and global variables defined within the same module will not be arranged so that variables that are accessed in the same function are close to each other
–no_mem_idioms	Makes the compiler not optimize certain memory access patterns
–do_explicit_zero_opt_in_named_sections	Disable the exception for variables in user-named sections, and thus treat explicit initializations to zero as zero initializations, not copy initializations
–require_prototypes	Force the compiler to verify that all functions have proper prototypes. Generates an error otherwise
–no_wrap_diagnostics	Does not wrap long lines in diagnostic messages
–diag_suppress=Pa050	Suppresses diagnostic message Pa050
-DS32K3XX	Predefine S32K3XX as a macro, with definition 1
-DS32K344	Predefine S32K344 as a macro, with definition 1
-DIAR	Predefine IAR as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode.
-DD_CACHE_ENABLE	Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initalization in source file system.↩ c under the Platform driver
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initalization in source file system.c under the Platform driver
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initalization in source file system.c under the Platform driver
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPO↩ RT as a macro, with definition 1. Allows drivers to be configured in user mode.

3.1.3.2 IAR Assembler Options

Assembler Option	Description
–cpu Cortex-M7	Targeted ARM processor for which IAR should generate the instruction set
–cpu_mode thumb	Selects the thumb mode for the assembler directive CODE
-g	Disables the automatic search for system include files
-r	Generates debug information

3.1.3.3 IAR Linker Options

Linker Option	Description
–map filename	Produces a map file
–config linkerfile	Use linkerfile as the linker script. This script replaces the default linker script (rather than adding to it)

Linker Option	Description
–cpu=Cortex-M7	Selects the ARM processor variant to link the application for
–fpu=FPv5-SP	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). Single-precision variant.
–entry _start	Treats _start as a root symbol and start label
–enable_stack_usage	Enables stack usage analysis. If a linker map file is produced, a stack usage chapter is included in the map file
–skip_dynamic_initialization	Dynamic initialization (typically initialization of C++ objects with static storage duration) will not be performed automatically during application startup
–no_wrap_diagnostics	Does not wrap long lines in diagnostic messages

3.2 Files required for compilation

This section describes the include files required to compile, assemble (if assembler code) and link the CAN driver. To avoid integration of incompatible files, all the include files from other modules shall have the same AR_MAJO↩ R_VERSION and AR_MINOR_VERSION, i.e. only files with the same AUTOSAR major and minor versions can be compiled.

3.2.0.0.1 CAN Driver Files:

- Can_TS_T40D34M20I0R0\src\Can.c
- Can_TS_T40D34M20I0R0\src\Can_Ipw.c
- Can_TS_T40D34M20I0R0\src\Can_Ipw_Irq.c
- Can_TS_T40D34M20I0R0\src\FlexCAN_Ip.c
- Can_TS_T40D34M20I0R0\src\FlexCAN_Ip_Irq.c
- Can_TS_T40D34M20I0R0\src\FlexCAN_Ip_Hw_Access.c
- Can_TS_T40D34M20I0R0\include\Can.h
- Can_TS_T40D34M20I0R0\include\Can_Flexcan_Types.h
- Can_TS_T40D34M20I0R0\include\Can_Ipw.h
- Can_TS_T40D34M20I0R0\include\Can_Ipw_Irq.h
- Can_TS_T40D34M20I0R0\include\Can_Ipw_Types.h
- Can_TS_T40D34M20I0R0\include\Can_Irq.h

- Can_TS_T40D34M20I0R0\include\FlexCAN_Ip.h
- Can_TS_T40D34M20I0R0\include\FlexCAN_Ip_DeviceReg.h

- Can_TS_T40D34M20I0R0\include\FlexCAN_Ip_Hw_Access.h
- Can_TS_T40D34M20I0R0\include\FlexCAN_Ip_Irq.h
- Can_TS_T40D34M20I0R0\include\FlexCAN_Ip_Types.h
- Can_TS_T40D34M20I0R0\include\FlexCAN_Ip_Wrapper.h
- Can_TS_T40D34M20I0R0\include\FlexCAN_Ip_TrustedFunctions.h

3.2.0.0.2 CAN Driver Generated Files (must be generated by the user using a configuration tool):

- Can_Cfg.h
- FlexCAN_Ip_Cfg.h
- Can_Ipw_Cfg.h
- Can_[VariantName]_PBcfg.c
- FlexCAN_Ip_[VariantName]_PBcfg.c
- Can_Ipw_[VariantName]_PBcfg.c
- Can_[VariantName]_PBcfg.h
- FlexCAN_Ip_[VariantName]_PBcfg.h
- Can_Ipw_[VariantName]_PBcfg.h

Note

3.3 Setting up the plugins

The CAN driver was designed to be configured by using the EB Tresos Studio (versionEB tresos Studio 27.1.0 or later.)

3.3.1 Location of various files inside the CAN module folder

VSMD (Vendor Specific Module Definition) file in EB tresos Studio XDM format:

- Can_TS_T40D34M20I0R0\config\Can.xdm

VSMD (Vendor Specific Module Definition) file(s) in AUTOSAR compliant EPD format:

- Can_TS_T40D34M20I0R0\autosar\Can_<subderivative_name>.epd

Chapter 4

Function calls to module

- Function Calls during Start-up
- Function Calls during Shutdown
- Function Calls during Wake-up

4.1 Function Calls during Start-up

The CAN module shall be initialized by Can_Init() service call during the start-up. API service Can_SetController↩ Mode(Can_Controller, CAN_CS_STARTED) shall be used for setting the CAN controller to running mode.

/note Can driver don't enable FlexCAN instances clocks, in order the user must enable the clocks from Clock Module Configuration for the instances used. Pin settings are not related to Can driver or plugin configuration. GPIO pins used for connection of CAN physical layer have to be properly assigned to the IPV_FlexCAN module prior the CAN initialization.

4.2 Function Calls during Shutdown

The IPV_FlexCAN IP has many Low Power Modes:

• Freeze Mode This low power mode is entered when the HALT and FRZ bits in the MCR Register are asserted. Module ignores the Rx input pin and drives the Tx pin as recessive, stops the prescaler, thus halting all CAN protocol activities and grants write access to the Error Counters Register (ECR), which is read-only in other modes. Exit from this mode is done by negating the FRZ and HALT bits in the MCR Register or when the MCU is removed from Debug Mode /note It is not possible to exit from this mode by receiving a message on the Can bus.

• Module Disable Mode This low power mode is entered when the MDIS bit in the MCR Register is asserted. Module shuts down the clocks to the CAN Protocol Interface and Message Buffer Management sub-modules. Exit from this mode is done by negating the MDIS bit in the MCR Register.

/note It is not possible to exit from this mode by receiving a message on the Can bus.

4.3 Function Calls during Wake-up

Hardware does not support wake-up.

Chapter 5

Module requirements

- Exclusive areas to be defined in BSW scheduler
- Exclusive areas not available on this platform
- Peripheral Hardware Requirements
- ISR to configure within AutosarOS - dependencies
- ISR Macro
- Other AUTOSAR modules - dependencies
- Data Cache Restrictions
- User Mode support
- Multicore support

5.1 Exclusive areas to be defined in BSW scheduler

CAN_EXCLUSIVE_AREA_00 is used in function Can_DisableControllerInterrupts to protect the variable Can_u8DisableInterruptLevel.

CAN_EXCLUSIVE_AREA_01 is used in function Can_EnableControllerInterrupts to protect the variable Can_u8DisableInterruptLevel.

CAN_EXCLUSIVE_AREA_02 is used in function Can_SetBaudrate to protect the register CAN_MCR in FlexCAN_EnterFreezeMode.

CAN_EXCLUSIVE_AREA_02 is used in function Can_SetControllerMode to protect the register CAN_MCR in FlexCAN_EnterFreezeMode.

CAN_EXCLUSIVE_AREA_02 is used in function Can_Init to protect the register CAN_MCR in FlexCA↩ N_EnterFreezeMode.

CAN_EXCLUSIVE_AREA_02 is used in function Can_DeInit to protect the register CAN_MCR in Flex↩ CAN_EnterFreezeMode.

CAN_EXCLUSIVE_AREA_02 is used in function DMA_Can_Callbackx to protect the register CAN_MCR in FlexCAN_EnterFreezeMode.

CAN_EXCLUSIVE_AREA_03 is used in function Can_ListenOnlyMode to protect the register CAN_MCR in FlexCAN_Enable.

CAN_EXCLUSIVE_AREA_03 is used in function Can_EnableControllerInterrupts to protect the register CAN_MCR in FlexCAN_Enable.

CAN_EXCLUSIVE_AREA_03 is used in function Can_DisableControllerInterrupts to protect the register CAN_MCR in FlexCAN_Enable.

CAN_EXCLUSIVE_AREA_03 is used in function Can_SetClockMode to protect the register CAN_MCR in FlexCAN_Enable.

CAN_EXCLUSIVE_AREA_04 is used in function Can_SetControllerMode to protect the register CAN_MCR in FlexCAN_ExitFreezeMode.

CAN_EXCLUSIVE_AREA_04 is used in function DMA_Can_Callbackx to protect the register CAN_MCR in FlexCAN_ExitFreezeMode.

CAN_EXCLUSIVE_AREA_05 is used in function Can_ListenOnlyMode to protect the register CAN_MCR in FlexCAN_Disable.

CAN_EXCLUSIVE_AREA_05 is used in function Can_EnableControllerInterrupts to protect the register CAN_MCR in FlexCAN_Disable.

CAN_EXCLUSIVE_AREA_05 is used in function Can_DisableControllerInterrupts to protect the register CAN_MCR in FlexCAN_Disable.

CAN_EXCLUSIVE_AREA_05 is used in function Can_SetControllerMode to protect the register CAN_MCR in FlexCAN_Disable.

CAN_EXCLUSIVE_AREA_05 is used in function Can_SetBaudrate to protect the register CAN_MCR in FlexCAN_Disable.

CAN_EXCLUSIVE_AREA_05 is used in function Can_Init to protect the register CAN_MCR in FlexCA↩ N_Disable.

CAN_EXCLUSIVE_AREA_05 is used in function Can_SetClockMode to protect the register CAN_MCR in FlexCAN_Disable.

CAN_EXCLUSIVE_AREA_05 is used in function Can_DeInit to protect the register CAN_MCR in Flex↩ CAN_Disable.

CAN_EXCLUSIVE_AREA_06 is used in function Can_EnableControllerInterrupts to protect the register CAN_MCR, CAN_CTRL1, CAN_CTRL2 in FlexCAN_SetErrIntCmd.

CAN_EXCLUSIVE_AREA_06 is used in function Can_DisableControllerInterrupts to protect the register CAN_MCR, CAN_CTRL1, CAN_CTRL2 in FlexCAN_SetErrIntCmd.

CAN_EXCLUSIVE_AREA_06 is used in function Can_SetControllerMode to protect the register CAN_↩ MCR, CAN_CTRL1, CAN_CTRL2 in FlexCAN_SetErrIntCmd.

CAN_EXCLUSIVE_AREA_07 is used in function Can_SetControllerMode to protect the register CAN_MCR in FlexCAN_Ip_SetStartMode.

CAN_EXCLUSIVE_AREA_08 is used in function Can_Init to protect the register CAN_MCR in FlexCA↩N_Ip_SetRxMaskType.

CAN_EXCLUSIVE_AREA_08 is used in function Can_SetControllerMode to protect the register CAN_MCR in FlexCAN_Ip_SetRxMaskType.

CAN_EXCLUSIVE_AREA_10 is used in function Can_ListenOnlyMode to protect the register CAN_CTRL1 in FlexCAN_Ip_SetListenOnlyMode.

CAN_EXCLUSIVE_AREA_10 is used in function Can_SetControllerMode to protect the register CAN_C↩TRL1 in FlexCAN_Ip_SetListenOnlyMode.

CAN_EXCLUSIVE_AREA_11 is used in function Can_AbortMb to protect the variable g_FlexCAN_u32↩ImaskBuff in FLEXCAN_ClearMsgBuffIntCmd.

CAN_EXCLUSIVE_AREA_11 is used in function Can_SetControllerMode to protect the variable g_Flex↩CAN_u32ImaskBuff in FLEXCAN_ClearMsgBuffIntCmd.

CAN_EXCLUSIVE_AREA_11 is used in function Can_ErrorIrqCallback to protect the variable g_FlexC↩AN_u32ImaskBuff in FLEXCAN_ClearMsgBuffIntCmd.

CAN_EXCLUSIVE_AREA_13 is used in function Can_Init to protect the register CAN_MCR in FlexCA↩N_SetRxFifoFilter.

CAN_EXCLUSIVE_AREA_13 is used in function Can_SetControllerMode to protect the register CAN_MCR in FlexCAN_SetRxFifoFilter.

CAN_EXCLUSIVE_AREA_14 is used in function Can_SetClockMode to protect the register CAN_CTRL1, CAN_EPRS in FlexCAN_Ip_SetBitrate.

CAN_EXCLUSIVE_AREA_14 is used in function Can_SetBaudrate to protect the register CAN_CTRL1, CAN_EPRS in FlexCAN_Ip_SetBitrate.

CAN_EXCLUSIVE_AREA_14 is used in function Can_SetControllerMode to protect the register CAN_C↩TRL1, CAN_EPRS in FlexCAN_Ip_SetBitrate.

CAN_EXCLUSIVE_AREA_15 is used in function Can_SetClockMode to protect the register CAN_MCR, CAN_FDCTRL, CAN_EDCBT, CAN_EPRS in FlexCAN_Ip_SetBitrateCbt.

CAN_EXCLUSIVE_AREA_15 is used in function Can_SetBaudrate to protect the register CAN_MCR, C↩AN_FDCTRL, CAN_EDCBT, CAN_EPRS in FlexCAN_Ip_SetBitrateCbt.

CAN_EXCLUSIVE_AREA_15 is used in function Can_SetControllerMode to protect the register CAN_↩MCR, CAN_FDCTRL, CAN_EDCBT, CAN_EPRS in FlexCAN_Ip_SetBitrateCbt.

CAN_EXCLUSIVE_AREA_16 is used in function Can_Init to protect the register CAN_FDCTRL, CAN↩_ETDC in FlexCAN_Ip_SetTDCOffset.

CAN_EXCLUSIVE_AREA_16 is used in function Can_SetBaudrate to protect the register CAN_FDCTRL, CAN_ETDC in FlexCAN_Ip_SetTDCOffset.

CAN_EXCLUSIVE_AREA_16 is used in function Can_SetControllerMode to protect the register CAN_F↩ DCTRL, CAN_ETDC in FlexCAN_Ip_SetTDCOffset.

CAN_EXCLUSIVE_AREA_17 is used in function Can_Init to protect the register CAN_CTRL2 in FlexC↩ AN_Ip_SetTxArbitrationStartDelay.

CAN_EXCLUSIVE_AREA_17 is used in function Can_SetBaudrate to protect the register CAN_CTRL2 in FlexCAN_Ip_SetTxArbitrationStartDelay.

CAN_EXCLUSIVE_AREA_17 is used in function Can_SetControllerMode to protect the register CAN_C↩ TRL2 in FlexCAN_Ip_SetTxArbitrationStartDelay.

CAN_EXCLUSIVE_AREA_18 is used in function Can_Write to protect the variable g_FlexCAN_u32↩ ImaskBuff in FlexCAN_SetMsgBuffIntCmd.

CAN_EXCLUSIVE_AREA_18 is used in function CANx_ORED_y_z_MB_IRQHandler to protect the variable g_FlexCAN_u32ImaskBuff in FlexCAN_SetMsgBuffIntCmd.

CAN_EXCLUSIVE_AREA_18 is used in function Can_SetControllerMode to protect the variable g_Flex↩ CAN_u32ImaskBuff in FlexCAN_SetMsgBuffIntCmd.

CAN_EXCLUSIVE_AREA_19 is used in function Can_Init to protect the register CAN_CTRL2 in FlexC↩ AN_ConfigTimestamp.

CAN_EXCLUSIVE_AREA_19 is used in function Can_SetControllerMode to protect the register CAN_C↩ TRL2 in FlexCAN_ConfigTimestamp.

Exclusive Areas implemented in Low level driver layer (IPL)
CAN_EXCLUSIVE_AREA_02 is used in function FlexCAN_EnterFreezeMode to protect the updates for:

- CAN_MCR register

  CAN_EXCLUSIVE_AREA_03 is used in function FlexCAN_Enable to protect the updates for:
- CAN_MCR register

  CAN_EXCLUSIVE_AREA_04 is used in function FlexCAN_ExitFreezeMode to protect the updates for:
- CAN_MCR register

  CAN_EXCLUSIVE_AREA_05 is used in function FlexCAN_Disable to protect the updates for:
- CAN_MCR register

  CAN_EXCLUSIVE_AREA_06 is used in function FlexCAN_SetErrIntCmd to protect the updates for:
- CAN_MCR, CAN_CTRL1, CAN_CTRL2 register

  CAN_EXCLUSIVE_AREA_07 is used in function FlexCAN_Ip_SetStartMode to protect the updates for:
- CAN_MCR register

  CAN_EXCLUSIVE_AREA_08 is used in function FlexCAN_Ip_SetRxMaskType to protect the updates for:
- CAN_MCR register

  CAN_EXCLUSIVE_AREA_09 is used in function FlexCAN_Ip_ClearTDCFail to protect the updates for:

- CAN_FDCTRL, CAN_ETDC register

  CAN_EXCLUSIVE_AREA_10 is used in function FlexCAN_Ip_SetListenOnlyMode to protect the updates for:

- CAN_CTRL1 register

  CAN_EXCLUSIVE_AREA_11 is used in function FLEXCAN_ClearMsgBuffIntCmd to protect the updates for:

- g_FlexCAN_u32ImaskBuff variable

  CAN_EXCLUSIVE_AREA_13 is used in function FlexCAN_SetRxFifoFilter to protect the updates for:

- CAN_MCR register

  CAN_EXCLUSIVE_AREA_14 is used in function FlexCAN_Ip_SetBitrate to protect the updates for:

- CAN_CTRL1, CAN_EPRS register

  CAN_EXCLUSIVE_AREA_15 is used in function FlexCAN_Ip_SetBitrateCbt to protect the updates for:

- CAN_MCR, CAN_FDCTRL, CAN_EDCBT, CAN_EPRS register

  CAN_EXCLUSIVE_AREA_16 is used in function FlexCAN_Ip_SetTDCOffset to protect the updates for:

- CAN_FDCTRL, CAN_ETDC register

  CAN_EXCLUSIVE_AREA_17 is used in function FlexCAN_Ip_SetTxArbitrationStartDelay to protect the updates for:

- CAN_CTRL2 register

  CAN_EXCLUSIVE_AREA_18 is used in function FlexCAN_SetMsgBuffIntCmd to protect the updates for:

- g_FlexCAN_u32ImaskBuff variable

  CAN_EXCLUSIVE_AREA_19 is used in function FlexCAN_ConfigTimestamp to protect the updates for:

- CAN_CTRL2 register

  CAN_EXCLUSIVE_AREA_20 is used in function FlexCAN_Ip_ManualBusOffRecovery to protect the updates for:

- CAN_CTRL1 register

Critical Region Exclusive Matrix Below is the table depicting the exclusivity between different critical region IDs from the CAN driver. If there is an "X" in a table, it means that those 2 critical regions cannot interrupt each other.

The critical regions from Tx/Rx interrupts are grouped in "ISRs Critical Regions (composed diagram)". If an exclusive area is "exclusive" with the composed "ISRs Critical Regions (composed diagram)" group, it means that it is exclusive with each one of the ISR critical regions.

Table 5.1 Exclusive Areas

#	AREA_00	AREA_01	AREA_02	AREA_03	AREA_04	AREA_05	AREA_06	AREA_07	AREA_08	AREA_09	AREA_10	AREA_11	AREA_13	AREA_14	AREA_15	AREA_16	AREA_17	AREA_18	AREA_19	AREA_20	ISR Critical Regions (composed diagram)
AREA_00		x																			
AREA_01	x																				
AREA_02				x	x	x	x	x	x				x		x						
AREA_03			x		x	x	x	x	x				x		x						
AREA_04			x	x		x	x	x	x				x		x						
AREA_05			x	x	x		x	x	x				x		x						

#	A↩R↩E↩A↩_↩00	A↩R↩E↩A↩_↩01	A↩R↩E↩A↩_↩02	A↩R↩E↩A↩_↩03	A↩R↩E↩A↩_↩04	A↩R↩E↩A↩_↩05	A↩R↩E↩A↩_↩06	A↩R↩E↩A↩_↩07	A↩R↩E↩A↩_↩08	A↩R↩E↩A↩_↩09	A↩R↩E↩A↩_↩10	A↩R↩E↩A↩_↩11	A↩R↩E↩A↩_↩13	A↩R↩E↩A↩_↩14	A↩R↩E↩A↩_↩15	A↩R↩E↩A↩_↩16	A↩R↩E↩A↩_↩17	A↩R↩E↩A↩_↩18	A↩R↩E↩A↩_↩19	A↩R↩E↩A↩_↩20	I↩S↩Rs Critical Regions (composed diagram)
A↩R↩E↩A↩_↩06			x	x	x	x		x	x		x		x	x	x		x		x	x	
A↩R↩E↩A↩_↩07			x	x	x	x	x		x				x		x						
A↩R↩E↩A↩_↩08			x	x	x	x	x	x					x		x						
A↩R↩E↩A↩_↩09															x	x					
A↩R↩E↩A↩_↩10							x							x						x	
A↩R↩E↩A↩_↩11																		x			

#	A↩ R↩ E↩ A↩ _↩ 00	A↩ R↩ E↩ A↩ _↩ 01	A↩ R↩ E↩ A↩ _↩ 02	A↩ R↩ E↩ A↩ _↩ 03	A↩ R↩ E↩ A↩ _↩ 04	A↩ R↩ E↩ A↩ _↩ 05	A↩ R↩ E↩ A↩ _↩ 06	A↩ R↩ E↩ A↩ _↩ 07	A↩ R↩ E↩ A↩ _↩ 08	A↩ R↩ E↩ A↩ _↩ 09	A↩ R↩ E↩ A↩ _↩ 10	A↩ R↩ E↩ A↩ _↩ 11	A↩ R↩ E↩ A↩ _↩ 13	A↩ R↩ E↩ A↩ _↩ 14	A↩ R↩ E↩ A↩ _↩ 15	A↩ R↩ E↩ A↩ _↩ 16	A↩ R↩ E↩ A↩ _↩ 17	A↩ R↩ E↩ A↩ _↩ 18	A↩ R↩ E↩ A↩ _↩ 19	A↩ R↩ E↩ A↩ _↩ 20	I↩ S↩ Rs Critical Regions (composed diagram)
A↩ R↩ E↩ A↩ _↩ 13			x	x	x	x	x	x	x						x						
A↩ R↩ E↩ A↩ _↩ 14							x				x				x					x	
A↩ R↩ E↩ A↩ _↩ 15			x	x	x	x	x	x	x	x			x	x		x					
A↩ R↩ E↩ A↩ _↩ 16										x					x						
A↩ R↩ E↩ A↩ _↩ 17							x												x		
A↩ R↩ E↩ A↩ _↩ 18												x									x

#	A↩ R↩ E↩ A↩ _↩ 00	A↩ R↩ E↩ A↩ _↩ 01	A↩ R↩ E↩ A↩ _↩ 02	A↩ R↩ E↩ A↩ _↩ 03	A↩ R↩ E↩ A↩ _↩ 04	A↩ R↩ E↩ A↩ _↩ 05	A↩ R↩ E↩ A↩ _↩ 06	A↩ R↩ E↩ A↩ _↩ 07	A↩ R↩ E↩ A↩ _↩ 08	A↩ R↩ E↩ A↩ _↩ 09	A↩ R↩ E↩ A↩ _↩ 10	A↩ R↩ E↩ A↩ _↩ 11	A↩ R↩ E↩ A↩ _↩ 13	A↩ R↩ E↩ A↩ _↩ 14	A↩ R↩ E↩ A↩ _↩ 15	A↩ R↩ E↩ A↩ _↩ 16	A↩ R↩ E↩ A↩ _↩ 17	A↩ R↩ E↩ A↩ _↩ 18	A↩ R↩ E↩ A↩ _↩ 19	A↩ R↩ E↩ A↩ _↩ 20	I↩ S↩ Rs Critical Regions (composed diagram)
A↩ R↩ E↩ A↩ _↩ 19							x										x				
A↩ R↩ E↩ A↩ _↩ 20							x				x			x							
I↩ S↩ Rs Critical Regions (composed diagram)																		x			

5.2 Exclusive areas not available on this platform

CAN_EXCLUSIVE_AREA_12 is not available.

5.3 Peripheral Hardware Requirements

peripheral hardware requirements template.

5.4 ISR to configure within AutosarOS - dependencies

The following ISR's are used by the CAN driver:

- Table with interrupts for S32K3XX

ISR Name	Hardware Interrupt Vector	S32K312	S32K314	S32K324	S32K344
CAN0_ORED_IRQHandler	125	YES	YES	YES	YES
CAN0_ORED_0_31_MB_IRQHandler	126	YES	YES	YES	YES
CAN0_ORED_32_63_MB_IRQHandler	127	YES	YES	YES	YES
CAN0_ORED_64_95_MB_IRQHandler	128	N/A	YES	YES	YES
CAN1_ORED_IRQHandler	129	YES	YES	YES	YES
CAN1_ORED_0_31_MB_IRQHandler	130	YES	YES	YES	YES
CAN1_ORED_32_63_MB_IRQHandler	131	YES	YES	YES	YES
CAN2_ORED_IRQHandler	132	YES	YES	YES	YES
CAN2_ORED_0_31_MB_IRQHandler	133	YES	YES	YES	YES
CAN2_ORED_32_63_MB_IRQHandler	134	YES	YES	YES	YES
CAN3_ORED_IRQHandler	135	YES	YES	YES	YES
CAN3_ORED_0_31_MB_IRQHandler	136	YES	YES	YES	YES
CAN4_ORED_IRQHandler	137	YES	YES	YES	YES
CAN4_ORED_0_31_MB_IRQHandler	138	YES	YES	YES	YES
CAN5_ORED_IRQHandler	139	YES	YES	YES	YES
CAN5_ORED_0_31_MB_IRQHandler	140	YES	YES	YES	YES

5.5 ISR Macro

RTD drivers use the ISR macro to define the functions that will process hardware interrupts. Depending on whether the OS is used or not, this macro can have different definitions.

5.5.1 Without an Operating System

The macro *USING_OS_AUTOSAROS* must not be defined.

5.5.1.1 Using Software Vector Mode

The macro *USE_SW_VECTOR_MODE* must be defined and the ISR macro is defined as:

```
#define ISR(IsrName) void IsrName(void)
```

In this case, the drivers' interrupt handlers are normal C functions and their prologue/epilogue will handle the context save and restore.

5.5.1.2 Using Hardware Vector Mode

The macro *USE_SW_VECTOR_MODE* must not defined and the ISR macro is defined as:

```
#define ISR(IsrName) INTERRUPT_FUNC void IsrName(void)
```

In this case, the drivers' interrupt handlers must also handle the context save and restore.

5.5.2 With an Operating System

Please refer to your OS documentation for description of the ISR macro.

5.6 Other AUTOSAR modules - dependencies

- **Base**: The Base module contains the common files/definitions needed by all MCAL modules.
- **Mcu**: The Mcu driver provides services for basic microcontroller initialization, power down functionality, reset and microcontroller specific functions required by other MCAL software modules. The clocks need to be initialized prior to using the Can driver. The clock frequency may affect the Can bit rate, the transmitting or receiving a Can frame process.\P
- **Port**: The Port module is used to configure the port pins with the needed modes, before they are used by the Can module.
- **EcuC** : The ECUC module is used for ECU configuration. Can modules need ECUC to retrieve the variant information.
- **Det** The Det module is used for enabling Default error detection. The API function used is Det_Report↩ Error(). The activation / deactivation of Default error detection is configurable using the 'CanDevErrorDetect' configuration parameter.
- **Resource**: Sub-Derivative model is selected from Resource configuration.
- **Rte**: The Rte module is needed for implementing data consistency of exclusive areas that are used by Can module.
- **EcuM**: This module is used for processing the Wakeup notifications of CAN. Whenever the module is in 'Sleep' mode and a wakeup event occurs, it is reported to EcuM through the EcuM_CheckWakeupEvent() API.
- **Mcl**: This module is used for enabling DMA channels for Can controllers.

5.7 Data Cache Restrictions

In the DMA transfer mode, DMA transfers may issue cache coherency problems. To avoid possible coherency issues when D-CACHE is enabled, the user shall ensure that the buffers used as TCD source and destination are allocated in the NON-CACHEABLE area (by means of Can_Memmap).

5.8 User Mode support

- User Mode configuration in the module
- User Mode configuration in AutosarOS

5.8.1 User Mode configuration in the module

The Can Driver can be run in user mode as long as the configuration parameter CanEnableUserModeSupport is enabled and MCAL_ENABLE_USER_MODE_SUPPORT is defined and call the following functions as trusted functions:

Function syntax	Description	Available via
void FlexCAN_ClearOutputLegacyFIFO(FLEXCAN_Type * base)	Clears Legacy RxFifo message buffers	FlexCAN_Ip_TrustedFunctions.h
void FlexCAN_ClearOutputEnhanceFIFO(FLEXCAN_Type * base)	Clears Enhanced RxFifo message buffers	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_Init_Privileged(uint8 Flexcan_Ip_u8Instance, Flexcan_Ip_StateType * Flexcan_Ip_pState, const Flexcan_Ip_ConfigType * Flexcan_Ip_pData)	Initializes the FlexCAN peripheral	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_ConfigRxFifo_Privileged(uint8 instance, Flexcan_Ip_RxFifoIdElementFormatType id_format, const Flexcan_Ip_IdTableType * id_filter_table)	FlexCAN Rx Legacy FIFO filter configuration	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_ConfigEnhancedRxFifo_Privileged(uint8 instance, const Flexcan_Ip_EnhancedIdTableType * id_filter_table)	FlexCAN Enhanced Rx FIFO filter configuration	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_MainFunctionBusOff_Privileged(uint8 instance)	Check a bus-off event	FlexCAN_Ip_TrustedFunctions.h
boolean FlexCAN_Ip_GetStopMode_Privileged(uint8 instance)	Check if the FlexCAN instance is STOPPED	FlexCAN_Ip_TrustedFunctions.h
boolean FlexCAN_Ip_GetStartMode_Privileged(uint8 instance)	Check if the FlexCAN instance is STARTED	FlexCAN_Ip_TrustedFunctions.h

Module requirements

Function syntax	Description	Available via
Flexcan_Ip_StatusType Flex↩ CAN_Ip_EnterFreezeMode_↩ Privileged(uint8 instance)	Enter FlexCAN Module in Freeze Mode	FlexCAN_Ip_Trusted↩ Functions.h
Flexcan_Ip_StatusType Flex↩ CAN_Ip_ExitFreezeMode_↩ Privileged(uint8 instance)	Exit FlexCAN Module from Freeze Mode	FlexCAN_Ip_Trusted↩ Functions.h
Flexcan_Ip_StatusType FlexC↩ AN_Ip_SetRxFifoGlobalMask_↩ Privileged(uint8 instance, uint32 mask)	Set RxFifo Global Mask	FlexCAN_Ip_Trusted↩ Functions.h
Flexcan_Ip_StatusType FlexCA↩ N_Ip_Deinit_Privileged(uint8 instance)	DeInitilize the FlexCAN instance driver	FlexCAN_Ip_Trusted↩ Functions.h
Flexcan_Ip_StatusType Flex↩ CAN_Ip_SetStartMode_↩ Privileged(uint8 instance)	Set the FlexCAN instance in STA↩ RT mode	FlexCAN_Ip_Trusted↩ Functions.h
Flexcan_Ip_StatusType Flex↩ CAN_Ip_SetStopMode_↩ Privileged(uint8 instance)	Set the FlexCAN instance in STOP mode	FlexCAN_Ip_Trusted↩ Functions.h
Flexcan_Ip_StatusType Flex↩ CAN_Ip_SetRxMaskType↩ _Privileged(uint8 instance, Flexcan_Ip_RxMaskType type)	Set RX masking type	FlexCAN_Ip_Trusted↩ Functions.h
Flexcan_Ip_StatusType Flex↩ CAN_Ip_SetRxMb14Mask_↩ Privileged(uint8 instance, uint32 mask)	Set Rx14Mask filter for message buffer 14	FlexCAN_Ip_Trusted↩ Functions.h
Flexcan_Ip_StatusType Flex↩ CAN_Ip_SetRxMb15Mask_↩ Privileged(uint8 instance, uint32 mask)	Set Rx14Mask filter for message buffer 15	FlexCAN_Ip_Trusted↩ Functions.h
Flexcan_Ip_StatusType FlexC↩ AN_Ip_SetRxIndividualMask↩ _Privileged(uint8 instance, uint8 mb_idx, uint32 mask)	Sets the FlexCAN Rx individual mask	FlexCAN_Ip_Trusted↩ Functions.h
Flexcan_Ip_StatusType FlexC↩ AN_Ip_SetRxMbGlobalMask_↩ Privileged(uint8 instance, uint32 mask)	Sets the FlexCAN Rx Message Buffer Global mask	FlexCAN_Ip_Trusted↩ Functions.h
Flexcan_Ip_StatusType FlexCA↩ N_Ip_SetBitrate_Privileged(uint8 instance, const Flexcan_Ip_Time↩ SegmentType ∗ bitrate, boolean enhExt)	Sets the FlexCAN bit rate for standard frames or the arbitration phase of FD frames	FlexCAN_Ip_Trusted↩ Functions.h
Flexcan_Ip_StatusType Flex↩ CAN_Ip_SetBitrateCbt_↩ Privileged(uint8 instance, const Flexcan_Ip_TimeSegmentType ∗ bitrate, boolean bitRateSwitch)	Sets the FlexCAN bit rate for the data phase of FD frames	FlexCAN_Ip_Trusted↩ Functions.h

Function syntax	Description	Available via
Flexcan_Ip_StatusType FlexCAN_Ip_SetTxArbitrationStart_Delay_Privileged(uint8 instance, uint8 value)	This function will set how many CAN bits the Tx arbitration process start point can	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_SetTDCOffset_Privileged(uint8 instance, boolean enable, uint8 offset)	Enables/Disables the Transceiver Delay Compensation feature and sets the Transceiver Delay Compensation Offset	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_EnableInterrupts_Privileged(uint8 u8Instance)	Enable all mb interrupts configured	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_DisableInterrupts_Privileged(uint8 u8Instance)	Disable all mb interrupts configured	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_SetErrorInt_Privileged(uint8 u8Instance, Flexcan_Ip_ErrorIntType type, boolean enable)	Enable\Disable Error or BusOff Interrupt	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_SetListenOnlyMode_Privileged(uint8 instance, const boolean enable)	Set FlexCAN Listen Only	FlexCAN_Ip_TrustedFunctions.h
Flexcan_Ip_StatusType FlexCAN_Ip_ConfigTimeStamp_Privileged(uint8 instance, const Flexcan_Ip_TimeStampConfigType * time_stamp)	Set FlexCAN Config Timestamp	FlexCAN_Ip_TrustedFunctions.h

5.8.2 User Mode configuration in AutosarOS

When User mode is enabled, the driver may has the functions that need to be called as trusted functions in AutosarOS context. Those functions are already defined in driver and declared in the header `<IpName>_Ip_TrustedFunctions.h`. This header also included all headers files that contains all types definition used by parameters or return types of those functions. Refer the chapter User Mode configuration in the module for more detail about those functions and the name of header files they are declared inside. Those functions will be called indirectly with the naming convention below in order to AutosarOS can call them as trusted functions.

```
Call_<Function_Name>_TRUSTED(parameter1,parameter2,...)
```

That is the result of macro expansion `OsIf_Trusted_Call` in driver code:

`#define` OsIf_Trusted_Call[1-6params](name,param1,...,param6) Call_##name##_TRUSTED(param1,...,param6)

So, the following steps need to be done in AutosarOS:

- Ensure `MCAL_ENABLE_USER_MODE_SUPPORT` macro is defined in the build system or somewhere global.

- Define and declare all functions that need to call as trusted functions follow the naming convention above in Integration/User code. They need to visible in `Os.h` for the driver to call them. They will do the marshalling of the parameters and call `CallTrustedFunction()` in OS specific manner.
- `CallTrustedFunction()` will switch to privileged mode and call `TRUSTED_<Function_Name>()`.
- `TRUSTED_<Function_Name>()` function is also defined and declared in Integration/User code. It will unmarshalling of the parameters to call `<Function_Name>`() of driver. The `<Function_Name>`() functions are already defined in driver and declared in `<IpName>_Ip_TrustedFunctions.h`. This header should be included in OS for OS call and indexing these functions.

See the sequence chart below for an example calling `Linflexd_Uart_Ip_Init_Privileged()` as a trusted function.

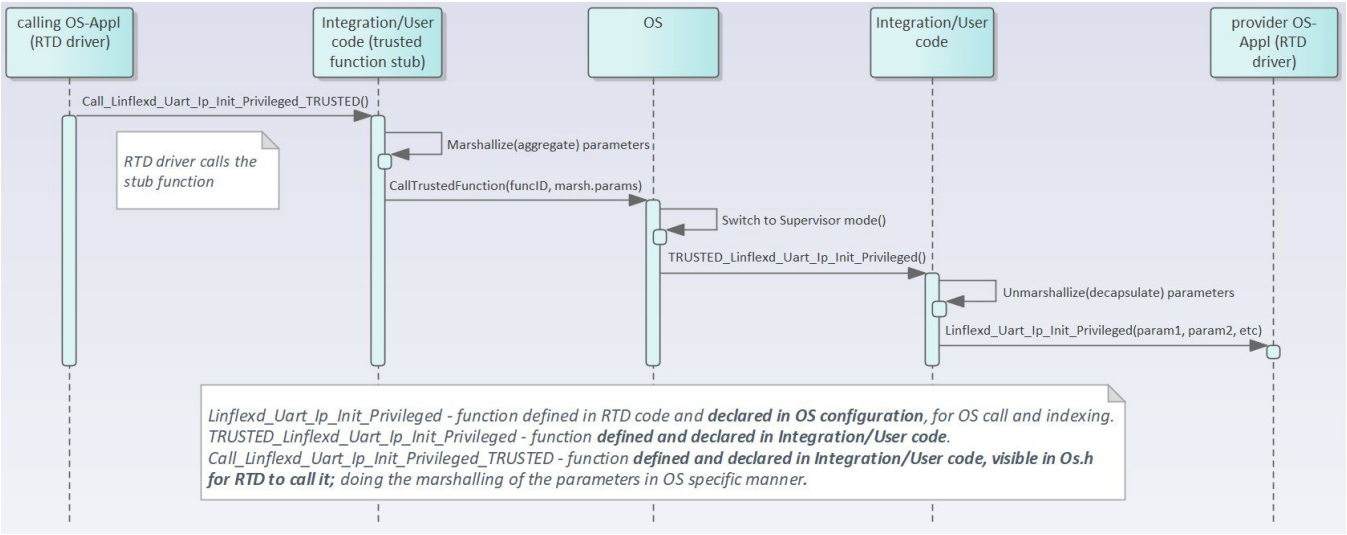


Figure 5.1 Example sequence chart for calling `Linflexd_Uart_Ip_Init_Privileged` as trusted function

5.9 Multicore support

The Can implements the "Autosar 4.4 MCAL Multicore Distribution" according to type II, in which the mappable element is set to HW Unit. For additional details, please refer to AUTOSAR_EXP_BSWDistributionGuide.
The Can and the mappable elements can be allocated to zero, one or several ECUC partitions, by means of "↩ CanEcucPartionRef". If the Can is mapped to zero ECUC partitions, the Can behavior reverts to single-core implementation, similar to previous Autosar versions. If the Can is mapped to one or more ECUC partitions, the Can enforces the following multi-core assumptions:
The Can assumes there is a single EcucPartition allocated per core. Internally, the module will use the Core ID returned by GetCoreID API to reference the appropriate global data and configuration elements.
The Can assumes the EcucCoreIDs are defined in a compact/consecutive order, starting from zero. The rationale is that the number of EcucPartitions is used for dimensioning the Can internal variables and the EcucCoreIDs are used for indexing those variables. (AR-86601 Zero based and dense IDs for OS-Cores and OSApplications) The Can assumes that initialization is performed on each core, Can_Init() is called separately for each core, using a different configguration structure. (Type II) The Can initialization expects the upper layer will pass the correct initialization pointer, specific to the partition in which the driver is to be used. For example: EcucPartition_1 is assigned to

CoreID 1; Can_Init function will be called with Can_Config_EcucPartition_1 configuration structure, on Core 1.
The Can will check upon each API call if the requested resource is configured to be available on the current core, if DET error reporting is enabled.
The Can requires that all variables in NonCacheable MemMap sections be allocated accordingly, to avoid data corruption in multicore context.
The Can assumes that RTE module implements the EXCLUSIVE AREAS to be core-aware only. The rationale is that the module implementation ensures data integrity by separating the mappable elements for different cores already, thus implementing the EXCLUSIVE AREAS in a blocking manner (ex: spin-lock) on a multicore scope, might affect the performance of the drivers on the two cores, although they might access separate HW elements. For single-core scope, the EXCLUSIVE AREAS keep the same purpose as on previous AUTOSAR implementations. (∗ - to be updated per Can usecase, to be detailed/removed if some modules require such kind of functionality for critical features which cannot be atomically shared among cores).
The Can assumes that each interrupt is routed by the system only to the core on which is supposed to be serviced.
The configuration structure name shall be available in the caller scope of Can_Init function by being declared with EXTERN, according to its generated name.

Chapter 6

Main API Requirements

- Main function calls within BSW scheduler
- API Requirements
- Calls to Notification Functions, Callbacks, Callouts

6.1 Main function calls within BSW scheduler

CAN Driver support 4 main functions that can be configured to be scheduled by BSW scheduler: • *void Can_↩ MainFunction_Write(void)*
• *void Can_MainFucntion_Read(void)*
• *void Can_MainFunction_BusOff(void)*
• *void Can_MainFunction_Mode(void)*
These Autosar APIs are scheduled if these 3 events are configured to be in “Polling” mode by the following parameters: • CanTxProcessing
#define CAN_TX_POLLING_SUPPORT (STD_ON)
• CanRxProcessing
#define CAN_RX_POLLING_SUPPORT (STD_ON)
• CanBusoffProcessing
#define CAN_BUSOFF_POLLING_SUPPORT (STD_ON)
The period for polling is configured by the following 4 parameters: • CanMainFunctionWritePeriod
#define CAN_MAINFUNCTION_PERIOD_WRITE (uint32)0.0010U
• CanMainFunctionReadPeriod
#define CAN_MAINFUNCTION_PERIOD_READ (uint32)0.0010U
• CanMainFunctionBusoffPeriod
#define CAN_MAINFUNCTION_PERIOD_BUSOFF (uint32)0.0010U
• CanMainFunctionModePeriod
#define CAN_MAINFUNCTION_MODE_PERIOD (uint32)0.0010U

Note

A configuration for an hardware unit can be possible in such a way that one controller will handle events by interrupts and another by polling method.

6.2 API Requirements

SWS_Can_00360,SWS_Can_00361,SWS_Can_00362,SWS_Can_00363,SWS_Can_00228,SWS_Can_00112,S↩
WS_Can_00185,SWS_Can_00235,

6.3 Calls to Notification Functions, Callbacks, Callouts

Call-back Notifications
The CAN stack provides the following call-back notifications:
• *CanIf_TxConfirmation*: This CAN Interface call-back function is called when a CAN message has been transmitted. *void CanIf_TxConfirmation(PduIdType CanTxPduId)*
• *CanIf_RxIndication*: This CAN Interface call-back function is called when valid CAN message is received. *void CanIf_RxIndication(const Can_HwType∗ Mailbox, const PduInfoType∗ PduInfoPtr)*
• *CanIf_ControllerBusOff*: This CAN Interface call-back function is called when the CAN controller reached the bus-off state (see CAN specification for further details). *void CanIf_ControllerBusOff(uint8 Controller)*
User Notification
• *void Can_RxTimeStampNotif(Can_HwHandleType Hoh, uint32 u32TimestampVal)*: The Rx TimeStamp Notification support for Received Objects is called when a message is received and Time Stamp feature for object is enabled. • *void Can_TxTimeStampNotif(Can_HwHandleType Hoh, uint32 CanTxPduId, uint32 u32Timestamp↩ Val)*: The Tx TimeStamp Notification support for Transmit Objects is called when a message is sent and Time Stamp feature for object is enabled.

Chapter 7

Memory allocation

- Sections to be defined in Can_MemMap.h
- Linker command file

7.1 Sections to be defined in Can_MemMap.h

Section name	Type of section	Description
CAN_START_SEC_CONFIG_DATA↩ _UNSPECIFIED	Configuration Data	Start of Memory Section for Config Data
CAN_STOP_SEC_CONFIG_DATA_↩ UNSPECIFIED	Configuration Data	End of Memory Section for Config Data
CAN_START_SEC_CODE	Code	Start of memory Section for Code
CAN_STOP_SEC_CODE	Code	End of memory Section for Code
CAN_START_SEC_VAR_CLEARED↩ _UNSPECIFIED	Variables	Used for variables, structures, arrays, when the SIZE (alignment) does not fit the criteria of 8,16 or 32 bit. These variables are initialized with values after every reset.
CAN_STOP_SEC_VAR_CLEARED_↩ UNSPECIFIED	Variables	End of above section.
CAN_START_SEC_VAR_CLEARED↩ _UNSPECIFIED	Variables	These variables are never cleared and never initialized by start-up code.
CAN_STOP_SEC_VAR_CLEARED_↩ UNSPECIFIED	Variables	End of above section.
CAN_START_SEC_CONST_UNSPE↩ CIFIED	Constant Data	Used for constants
CAN_STOP_SEC_CONST_UNSPECI↩ FIED	Constant Data	End of above section.
CAN_START_SEC_VAR_CLEARED↩ _UNSPECIFIED_NO_CACHEABLE	Variables	Used for variables, structures, arrays when the SIZE (alignment) does not fit the criteria of 8, 16 or 32 bit. Normally, this section is used to store descriptors and data buffers. This section must also be cache inhibited
CAN_STOP_SEC_VAR_CLEARED_↩ UNSPECIFIED_NO_CACHEABLE	Variables	End of the above section

7.2 Linker command file

Memory shall be allocated for every section defined in the driver's "<Module>"_MemMap.h.

Chapter 8

Integration Steps

This section gives a brief overview of the steps needed for integrating this module:

1. Generate the required module configuration(s). For more details refer to section Files Required for Compilation
2. Allocate the proper memory sections in the driver's memory map header file ("<Module>"_MemMap.h) and linker command file. For more details refer to section Sections to be defined in <Module>_MemMap.h
3. Compile & build the module with all the dependent modules. For more details refer to section Building the Driver

Chapter 9

External assumptions for driver

The section presents requirements that must be complied with when integrating the CAN driver into the application.

External Assumption Req ID	External Assumption Text
SWS_Can_00436	Can_GeneralTypes.h shall contain all types and constants that are shared among the AUTOSAR CAN modules Can, CanIf and CanTrcv. Note↩ : Implemented in base
SWS_Can_00415	Name: - Can_PduType - Type: - Structure - Element: - PduIdType - sw↩ PduHandle - – - uint8 - length - – - Can_IdType - id - – - uint8∗ - sdu - – - Description: - This type unites PduId (swPduHandle), SduLength (length), SduData (sdu), and CanId (id) for any CAN L-SDU. - Available via: - Can↩ _GeneralTypes.h - Note: defined into Can_GeneralTypes.h included into Base module
SWS_Can_00416	Name: - Can_IdType - Type: - uint32 - Range: - Standard32Bit - – - 0..0x400007FF - Extended32Bit - – - 0..0xDFFFFFFF - Description: - Represents the Identifier of an L-PDU. The two most significant bits specify the frame type: 00 CAN message with Standard CAN ID 01 CAN FD frame with Standard CAN ID 10 CAN message with Extended CAN ID 11 CAN FD frame with Extended CAN ID - Available via: - Can_GeneralTypes.h - Note: defined into Can_GeneralTypes.h included into Base module
SWS_Can_00429	Name: - Can_HwHandleType - Type: - uint8, uint16 - Range: - Standard - – - 0..0x0FF - Extended - – - 0..0xFFFF - Description: - Represents the hardware object handles of a CAN hardware unit. For CAN hardware units with more than 255 HW objects use extended range. - Available via: - Can↩ _GeneralTypes.h - Note: defined into Can_GeneralTypes.h included into Base module
SWS_Can_00039	Range: - - 0x02 - transmit request could not be processed because no transmit object was available - Description: - Overlayed return value of Std↩ _ReturnType for CAN driver API Can_Write() - Available via: - Can↩ _GeneralTypes.h - Note: defined into Can_GeneralTypes.h included into Base module
SWS_Can_00024	The valid values that can be configured are hardware dependent. Therefore the rules and constraints can't be given in the standard. The configuration tool is responsible to do a static configuration checking, also regarding dependencies between modules (i.e. Port driver, MCU driver etc.)

External assumptions for driver

External Assumption Req ID	External Assumption Text
SWS_Can_91013	Name: - Can_ControllerStateType - Type: - Enumeration - Range: - CA↩ N_CS_UNINIT - 0x00 - CAN controller state UNINIT. - CAN_CS_ST↩ ARTED - 0x01 - CAN controller state STARTED. - CAN_CS_STOPPED - 0x02 - CAN controller state STOPPED. - CAN_CS_SLEEP - 0x03 - CAN controller state SLEEP. - Description: - States that are used by the several ControllerMode functions. - Available via: - Can_GeneralTypes.h - Note: defined into Can_GeneralTypes.h included into Base module
SWS_Can_91003	Name: - Can_ErrorStateType - Type: - Enumeration - Range: - CAN↩ _ERRORSTATE_ACTIVE - – - The CAN controller takes fully part in communication. - CAN_ERRORSTATE_PASSIVE - – - The CAN controller takes part in communication, but does not send active error frames. - CAN_ERRORSTATE_BUSOFF - – - The CAN controller does not take part in communication. - Description: - Error states of a CAN controller. - Available via: - Can_GeneralTypes.h - Note: defined into Can_General↩ Types.h included into Base module
SWS_CAN_00496	Name: Can_HwType Type: Structure Element: Can_IdType: CanId-Standard/Extended CAN ID of CAN L-PDU. Can_HwHandleType: Hoh - ID of the corresponding Hardware Object Range. uint8: ControllerId - ControllerId provided by CanIf clearly identify the corresponding controller Description: This type defines a data structure which clearly provides an Hardware Object Handle including its corresponding CAN Controller and therefore CanDrv as well as the specific CanId. Available via: Can_↩ GeneralTypes.h
EA_RTD_00001	The external application shall call Can_Init function only when the driver state is CAN_UNINIT and the state of all controllers is UNINIT.
EA_RTD_00002	The external application shall call Can_MainFunction_Write only after driver initialization.
EA_RTD_00003	The external application shall call Can_MainFunction_Read only after driver initialization.
EA_RTD_00004	The external application shall call Can_MainFunction_BusOff only after driver initialization.
EA_RTD_00005	The external application shall call Can_MainFunction_Wakeup only after driver initialization.
EA_RTD_00006	The external application shall call Can_SetControllerMode only after driver initialization.
EA_RTD_00007	The external application shall call Can_DisableControllerInterrupts function only after driver initialization.
EA_RTD_00008	The external application shall call Can_EnableControllerInterrupts function only after driver initialization.
EA_RTD_00009	The external application shall call Can_Write function only after driver initialization.
EA_RTD_00010	The external application shall assure that Can_Init does not preempt and is not preempted by any other CAN driver API excepting Can_GetVersion↩ Info. The external application shall assure that Can_Init does not preempt itself.
EA_RTD_00011	The external application shall assure that Can_MainFunction_Write does not preempt and is not preempted by any other CAN driver API exception Can_GetVersionInfo.The external application shall assure that Can↩ _MainFunction_Write does not preempt itself.

External Assumption Req ID	External Assumption Text
EA_RTD_00012	The external application shall assure that Can_MainFunction_Read does not preempt and is not preempted by any other CAN driver API exception Can_GetVersionInfo.The external application shall assure that Can_MainFunction_Read does not preempt itself.
EA_RTD_00013	The external application shall assure that Can_MainFunction_BusOff does not preempt and is not preempted by any other CAN driver API exception Can_GetVersionInfo.The external application shall assure that Can_MainFunction_BusOff does not preempt itself.
EA_RTD_00014	The external application shall assure that Can_SetControllerMode does not preempt and is not preempted by any other CAN driver API using the same controller parameter. The external application shall assure that Can_SetControllerMode does not preempt itself.
EA_RTD_00015	The external application shall assure that Can_DisableControllerInterrupts does not preempt and is not preempted by any other CAN driver API using the same controller parameter.
EA_RTD_00016	The external application shall assure that Can_EnableControllerInterrupts does not preempt and is not preempted by any other CAN driver API using the same controller parameter.
EA_RTD_00017	The external application shall assure that Can_Write does not preempt and is not preempted by any other CAN driver API using the same controller as the hardware handle parameter.The external application shall assure that Can_Write does not preempt itself for the same hardware handle parameter.
EA_RTD_00018	The external application shall call Can_ChangeBaudrate only when the CAN controller is in state STOPPED.
EA_RTD_00019	The external application shall call Can_Init function only when the driver state is CAN_UNINIT and the state of all controllers is UNINIT.
EA_RTD_00020	The external application shall assure that Can_CheckWakeup does not preempt and is not preempted by any other CAN driver API using the same controller parameter. The external application shall assure that Can_CheckWakeup does not preempt itself.
EA_RTD_00021	The external application shall call Can_CheckWakeup function only after driver initialization.
EA_RTD_00022	The external application shall call Can_MainFunction_Mode function only after driver initialization.
EA_RTD_00023	The external application shall call Can_Init function only when the driver state is CAN_UNINIT and the state of all controllers is UNINIT.
EA_RTD_00024	The external application shall call Can_SetControllerMode(CAN_T_START) only when the CAN controller is in state STOPPED.
EA_RTD_00025	The external application shall call Can_SetControllerMode(CAN_T_STOP) only when the CAN controller is in state STARTED or STOPPED.
EA_RTD_00026	The external application shall call Can_SetControllerMode(CAN_T_SLEEP) only when the CAN controller is in state SLEEP or STOPPED.
EA_RTD_00027	The external application shall call Can_SetControllerMode(CAN_T_WAKEUP) only when the CAN controller is in state SLEEP or STOPPED.
EA_RTD_00028	The external application shall assure that Can_ChangeBaudrate does not preempt and is not preempted by any other CAN driver API using the same controller parameter.The external application shall assure that Can_ChangeBaudrate does not preempt itself.The external application shal call Can_ChangeBaudrate only when the controller parameter is STOPPED.

External assumptions for driver

External Assumption Req ID	External Assumption Text
EA_RTD_00029	The external application shall call Can_ChangeBaudrate only after driver initialization and when the configured controller is in the STOPPED state.
EA_RTD_00030	The external application shall assure that Can_CheckBaudrate does not preempt and is not preempted by any other CAN driver API using the same controller parameter.
EA_RTD_00031	The external application shall call Can_CheckBaudrate only after driver initialization.
EA_RTD_00071	If interrupts are locked, a centralized function pair to lock and unlock interrupts shall be used.
EA_RTD_00081	The integrator shall assure that <MSN>_Init() and <MSN>_DeInit() functions do not interrupt each other.
EA_RTD_00082	When caches are enabled and data buffers are allocated in cacheable memory regions the buffers involved in DMA transfer shall be aligned with both start and end to cache line size. Note: **Rationale**: This ensures that no other buffers/variables compete for the same cache lines.
EA_RTD_00092	The integrator shall allocate a single EcucPartition per core or the partition in which the Can is allocated shall be exclusively mapped to a core. Note: Internally, the Can will use the Core ID returned by GetCoreID API to reference the appropriate global data and configuration elements, that is why a core should reference only one configured partition.
EA_RTD_00093	The application shall define EcucCoreIDs in a compact/consecutive order, starting from zero.
EA_RTD_00094	When multicore support is enabled, the application shall call Can_Init() for each core, using the dedicated configuration pointer for that core.
EA_RTD_00096	The application shall pass the correct initialization pointer, specific to the partition in which the driver is to be used.
EA_RTD_00106	Standalone IP configuration and HL configuration of the same driver shall be done in the same project
EA_RTD_00107	The integrator shall use the IP interface only for hardware resources that were configured for standalone IP usage. Note: The integrator shall not directly use the IP interface for hardware resources that were allocated to be used in HL context.
EA_RTD_00108	The integrator shall use the IP interface to a build a CDD, therefore the BSWMD will not contain reference to the IP interface
EA_RTD_00109	Name: Can_TimeStampType Type: Structure Element: uint32 nanoseconds Nanoseconds part of the time uint32 seconds Seconds part of the time Description: Shall define time stamps types based on relative time. Value range: - Seconds: 0 .. 4.294.967.295 s (136 years) - Nanoseconds: 0 .. 999.↩ 999.999 ns Available via Can_GeneralTypes.h

How to Reach Us:

Home Page:
nxp.com

Web Support:
nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, AltiVec, C–5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C–Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Qonverge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2022 NXP B.V.